Session 4 : Tools, MCP and Agentic Workflows I

0. Setup

```
%pip install -q anthropic
```

```
import os, getpass, json, datetime
if not os.environ.get("ANTHROPIC_API_KEY"):
    os.environ["ANTHROPIC_API_KEY"] = getpass.getpass("Paste your Anthropic API
key: ")

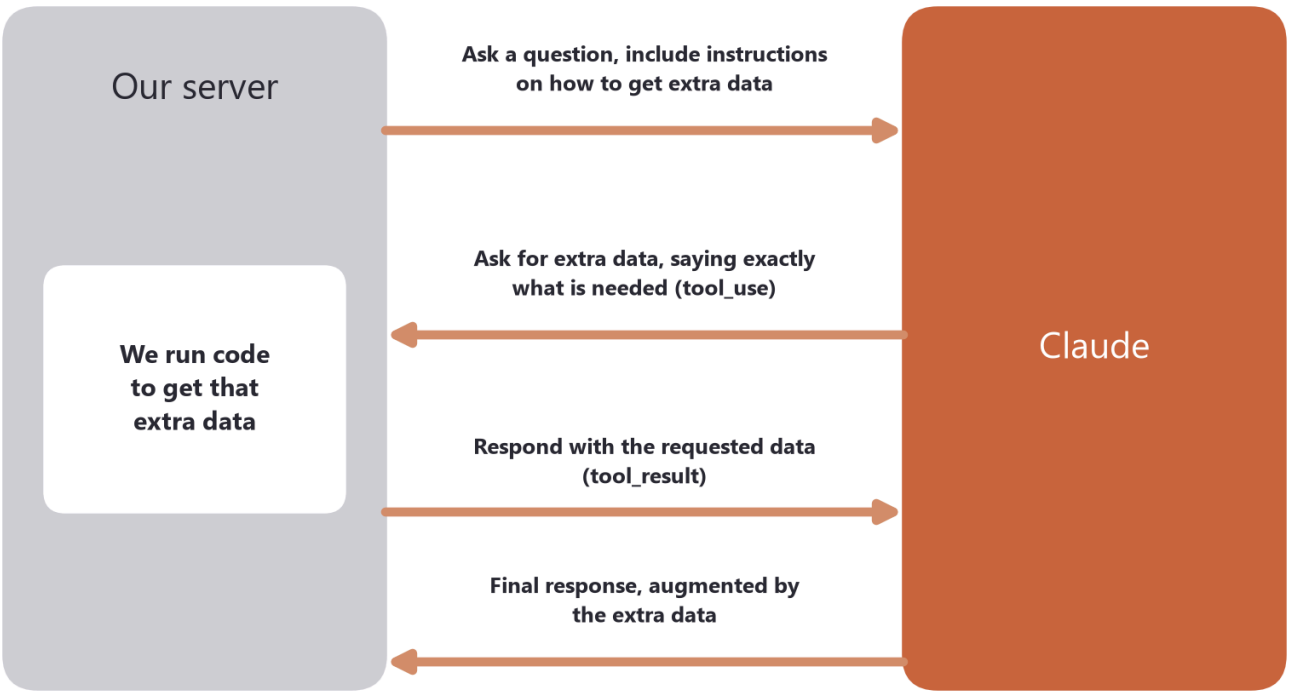
from anthropic import Anthropic

client = Anthropic()
MODEL = "claude-sonnet-5-5"      # tool use is more reliable on a stronger model

def text_of(message):
    return "".join(b.text for b in message.content if b.type == "text")
```

1. Tool Use

Claude **cannot run your code**. It asks **you** to, by returning a `tool_use` block. You run the function and send the answer back.



A tool definition

Three parts: `name`, `description`, `input_schema` (JSON Schema). **The description is the biggest lever.** Say what it does, **when** to use it, what it returns.

```
get_weather = {
    "name": "get_weather",
    "description": "Gets the current weather for a city. Use when the user asks
about current "
                   "conditions. Returns temperature in Celsius and a short
summary. No forecasts.",
    "input_schema": {
        "type": "object",
        "properties": {"city": {"type": "string", "description": "City name, e.g.
'Chennai'"}},
        "required": ["city"],
    },
}

def get_weather_impl(city):
    return {"city": city, "temp_c": 31, "summary": "humid, light cloud"}     #
pretend API
```

The round trip

```
messages = [{"role": "user", "content": "What's the weather in Chennai?"}]
resp = client.messages.create(model=MODEL, max_tokens=1000, tools=[get_weather],
messages=messages)
print("stop_reason:", resp.stop_reason)
for b in resp.content:
    print(b.type, getattr(b, "input", ""))
```

`stop_reason` is `tool_use`. Now run the tool and reply:

```
messages.append({"role": "assistant", "content": resp.content})          # the
FULL reply
tu = next(b for b in resp.content if b.type == "tool_use")
messages.append({"role": "user", "content": [
    {"type": "tool_result", "tool_use_id": tu.id, "content":
json.dumps(get_weather_impl(**tu.input))}
]})
final = client.messages.create(model=MODEL, max_tokens=1000, tools=[get_weather],
messages=messages)
print(text_of(final))
```

Rules the exam tests

Rule	Why
Append the **whole** `response.content`, not just text	The `tool_use` block must precede its result
`tool_result` goes in a `user` message	Roles alternate
`tool_use_id` must **match** the `tool_use id`	Pairs request and result
Send `tools` on **every** request	Claude needs the schema to read the history
Check `stop_reason == "tool_use"`	Text can appear alongside a tool call

`tool_choice` and strict

`tool_choice`	Behaviour
`auto`	Claude decides (default)
`any` / `tool`	Must use a tool (check the model: rejected on the newest models)
`none`	No tools

Add `"strict": true` to a tool so `input` is **guaranteed** to match the schema.

Quiz. After Claude returns `stop_reason: "tool_use"`, where does your `tool_result` go?

```
[( )] In an `assistant` message
[(X)] In a `user` message
[( )] In the `system` prompt
[( )] In the `tools` list
***

The result is your reply to Claude's request, so it is a user turn.

***
```

2. Tool Design

Error returns

When a tool fails, **tell Claude**. Return a `tool_result` with `is_error: true` and a message it can act on. Claude can retry with new arguments or explain the problem.

```python
def run_tool(block):
    try:
        if block.name == "get_weather":
            if block.input["city"].lower() not in ("chennai", "mumbai", "delhi"):
                raise ValueError(f"No data for '{block.input['city']}'. Try
Chennai, Mumbai or Delhi.")
            out = get_weather_impl(**block.input)
        elif block.name == "get_current_datetime":
```

```
            out = {"now": datetime.date.today().isoformat()}
        elif block.name == "add_days":
            d = datetime.date.fromisoformat(block.input["date"]) +
datetime.timedelta(days=block.input["days"])
            out = {"date": d.isoformat(), "weekday": d.strftime("%A")}
        else:
            raise ValueError(f"Unknown tool {block.name}")
        return {"type": "tool_result", "tool_use_id": block.id, "content":
json.dumps(out)}
    except Exception as e:
        return {"type": "tool_result", "tool_use_id": block.id, "content": str(e),
"is_error": True}
```

Parallel calls

Claude may return **several** `tool_use` blocks in one reply. Run them all and send **all** results in **one** `user` message.

Designing the tool set

- Few, clear tools beat many overlapping ones.
- Names are verbs: `get_weather`, `add_days`.
- Describe **when not** to use a tool.
- Tools should be abstract enough to be reusable, not one per tiny action.

Quiz. A tool throws an exception. What is the best handling?

```
[( )] Crash the program
[( )] Silently return an empty string
[(X)] Return a `tool_result` with `is_error: true` and a helpful message
[( )] Stop calling the API
***

Claude can recover from an error it can read.

***
```

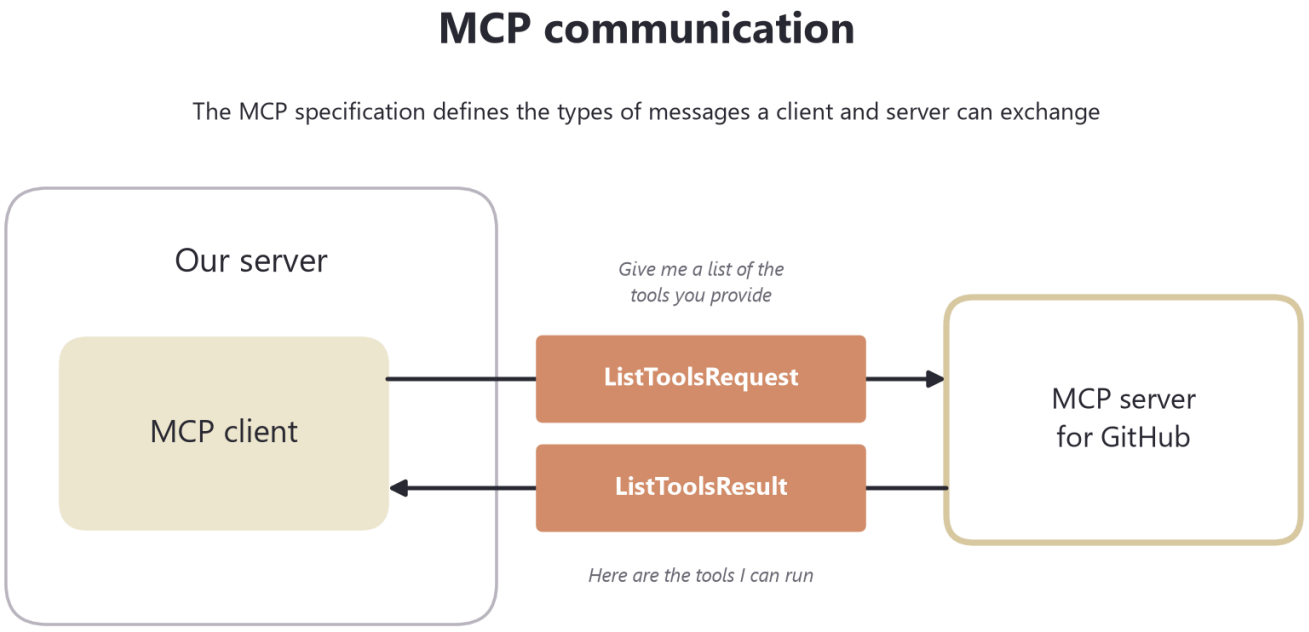
3. MCP Architecture

Model Context Protocol (MCP) is an open standard so any app can use any tool server, without writing custom glue for every pair.

Role	What it is
Host	The app the user runs (Claude Desktop, Claude Code, your agent)
Client	Lives in the host, one connection per server
Server	Exposes capabilities. Local (stdio) or remote (HTTP).

Transport	Use
stdio	Local server started as a subprocess
Streamable HTTP	Remote servers

Primitive	Controlled by	Meaning
Tools	The model	Actions Claude can call
Resources	The app	Read-only data (files, records)
Prompts	The user	Reusable prompt templates



A tiny MCP server (needs `pip install mcp`; test with `mcp dev server.py`):

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Docs")
docs = {"plan.md": "The plan outlines the project steps."}

@mcp.tool()
def read_doc(doc_id: str) -> str:
    """Read a document by id."""
    return docs[doc_id]

@mcp.resource("docs://list")
def list_docs() -> list[str]:
    return list(docs)

@mcp.prompt()
def summarise(doc_id: str) -> str:
    return f"Summarise document {doc_id}."
```

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Security: an MCP server is code you run. Only install servers you trust, and give each the least access it needs.

Quiz. An MCP **resource** is best described as:

```
[( )] A function Claude calls to take an action
[(X)] Read-only data exposed by the server
[( )] A prompt template the user triggers
[( )] A network transport
***

Tools act, resources are data, prompts are templates.

***
```

4. Hands-On Lab: Build a Tool Loop

A loop keeps going **until Claude stops asking for tools**.

```
datetime_tool = {"name": "get_current_datetime", "description": "Returns today's
date as YYYY-MM-DD.",
                 "input_schema": {"type": "object", "properties": {}}}
add_days_tool = {"name": "add_days",
                 "description": "Adds a number of days to a date. Returns the new
date and weekday.",
                 "input_schema": {"type": "object",
                                  "properties": {"date": {"type": "string",
"description": "YYYY-MM-DD"},
                                                 "days": {"type": "integer"}},
                                  "required": ["date", "days"]}}
TOOLS = [get_weather, datetime_tool, add_days_tool]

def run_conversation(user_text, tools=TOOLS, max_turns=8):
    messages = [{"role": "user", "content": user_text}]
    for turn in range(max_turns):                                  # stop
condition
        resp = client.messages.create(model=MODEL, max_tokens=1000, tools=tools,
messages=messages)
        messages.append({"role": "assistant", "content": resp.content})
        if resp.stop_reason != "tool_use":
            return text_of(resp)
        results = []
        for b in resp.content:
            if b.type == "tool_use":
                print(f"  turn {turn}: {b.name}({b.input})")
                results.append(run_tool(b))
        messages.append({"role": "user", "content": results})
```

```
    return "Stopped: max_turns reached"

print(run_conversation("What day of the week is 103 days from today?"))
print(run_conversation("Compare the weather in Chennai and Mumbai."))
print(run_conversation("What's the weather in Atlantis?"))
```

Tasks

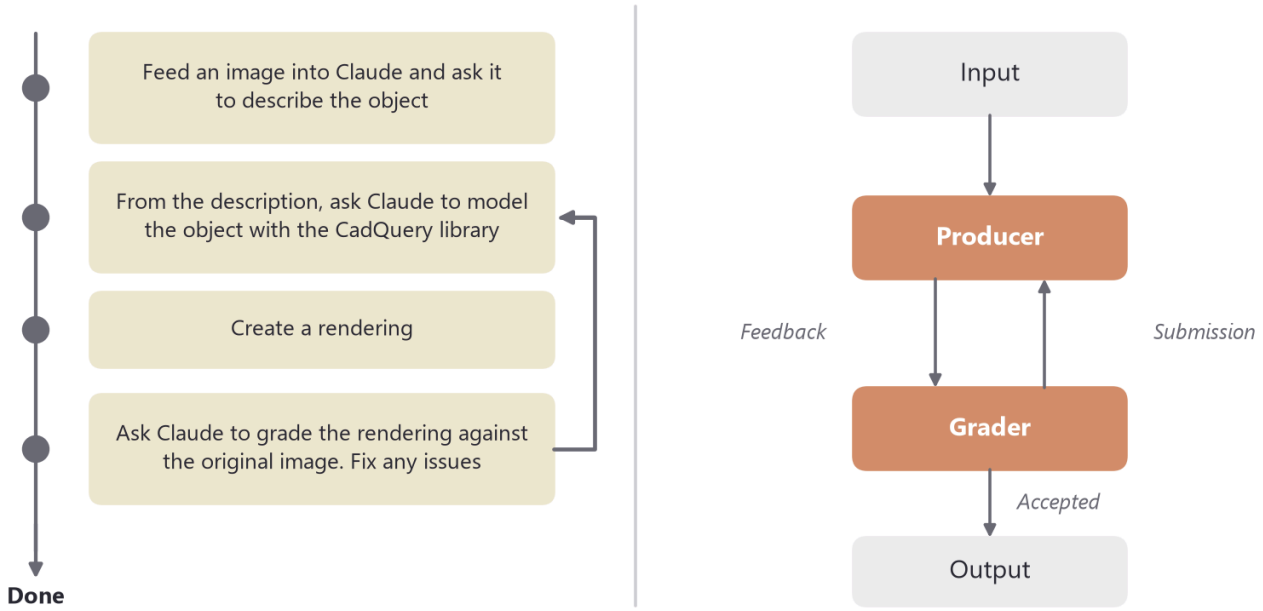
- [ ] Which prompt produced **parallel** calls?
- [ ] What did Claude do after the `is_error` result for Atlantis?
- [ ] Set `max_turns=1`. What happens?

5. Workflow Patterns

Workflow: the *code* decides the steps. **Agent:** the *model* decides the steps. Start with the simplest thing that works.

Pattern	Idea	Use when
Prompt chaining	Output of step 1 feeds step 2	A task splits into fixed stages
Routing	Classify, then send to a specialised path	Different inputs need different handling
Parallelization	Run independent calls at once (split work, or vote)	Speed, or confidence from several answers
Orchestrator-workers	A lead model plans subtasks and delegates	Subtasks are **not known in advance**
Evaluator-optimizer	One model generates, another critiques, loop	Clear quality criteria, iteration helps

Workflow type: evaluator-optimizer




```
def ask(prompt, system=None, model="claude-haiku-5-5"):
    p = {"model": model, "max_tokens": 1000, "messages": [{"role": "user",
"content": prompt}]}
    if system:
        p["system"] = system
    return text_of(client.messages.create(**p)).strip()

# 1) Chaining: outline, then write
outline = ask("Give a 3-bullet outline for a short post about prompt caching.")
post = ask(f"Write a 60-word post from this outline:\n{outline}")
print(post, "\n")

# 2) Routing: classify, then pick a specialist
kind = ask("Reply one word, 'billing' or 'technical': My payment failed
twice").lower()
experts = {"billing": "You are a billing specialist. Be brief.", "technical": "You
are a support engineer. Be brief."}
print(kind, "->", ask("My payment failed twice", system=experts.get(kind,
experts["technical"])))
```

```
# 3) Parallelization: three independent calls at once
from concurrent.futures import ThreadPoolExecutor

topics = ["caching", "batching", "streaming"]
with ThreadPoolExecutor() as pool:
    for t, a in zip(topics, pool.map(lambda t: ask(f"One sentence: why use {t}
with the Claude API?"), topics)):
        print(f"{t}: {a}")
```

```
# 5) Evaluator-optimizer: generate, critique, improve once
draft = ask("Write a one-sentence tagline for a note-taking app.")
critique = ask(f"Critique this tagline in one sentence. Is it specific and
memorable?\n{draft}")
better = ask(f"Improve the tagline using the critique.\nTagline:
{draft}\nCritique: {critique}\nReturn only the tagline.")
print("\nDRAFT :", draft, "\nBETTER:", better)
```

Choosing: fixed steps, use chaining. Different input types, use routing. Unknown subtasks, orchestrator-workers. Need quality loops, evaluator-optimizer. Open-ended, long tasks, an **agent**.

Quiz. Subtasks depend on what the model finds and cannot be listed in advance. Which pattern?

```
[( )] Prompt chaining
[( )] Routing
[(X)] Orchestrator-workers
[( )] A single prompt with examples
***

The lead model decides the subtasks at run time.

***
```

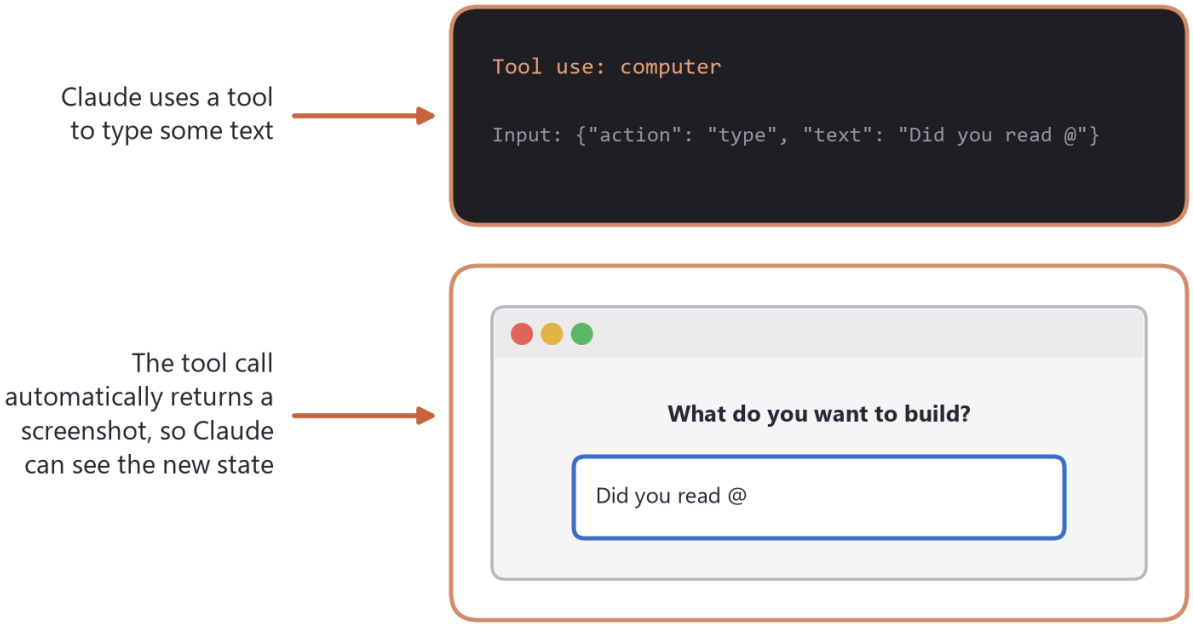
6. Agent Control

An **agent** is an LLM using tools in a loop, deciding its own next step. More freedom means more risk, so **add control**.

Control	Purpose
Stop conditions	Task done (`end_turn`), `max_turns`, a budget (tokens, time, cost), or a blocked tool
Environment inspection	After each action, the agent checks the **real result** (file contents, test output), not its assumption
Human-in-the-loop	A person approves risky or irreversible actions
Least privilege	Only the tools the task needs. Read-only by default.
Logging	Record every tool call so a failed run can be debugged

Environment inspection

Claude acts blind: it needs a way to observe what its action did



Add an approval gate to the loop. Safe tools run automatically, risky ones ask first.

```
RISKY = {"send_email"}
AUTO_APPROVE = True      # set False to be asked in the notebook

def approved(name, args):
    if name not in RISKY or AUTO_APPROVE:
        return True
    return input(f"Allow {name}({args})? [y/N] ").strip().lower() == "y"

email_tool = {"name": "send_email", "description": "Sends an email.
Irreversible.",
              "input_schema": {"type": "object", "properties": {"to": {"type":
"string"}, "body": {"type": "string"}},
                               "required": ["to", "body"]}}

def guarded_run(block):
    if not approved(block.name, block.input):
        return {"type": "tool_result", "tool_use_id": block.id, "content": "Denied
by the user.", "is_error": True}
    if block.name == "send_email":
        return {"type": "tool_result", "tool_use_id": block.id, "content": "Email
queued."}
    return run_tool(block)

print("Gate for a risky tool:", approved("send_email", {"to": "x@y.com"}), "| for
a safe tool:", approved("add_days", {}))
```

Quiz. Which is the **best** protection for an agent that can delete files? *(select one)*

```
[( )] Tell it in the prompt to be careful
[(X)] Require human approval for delete, and scope the tool to one folder
[( )] Raise `max_tokens`
[( )] Use a bigger model
***

Enforce limits in code and with permissions. Prompts are not a guarantee.

***
```

7. Agent SDK Lab

The **Claude Agent SDK** runs Claude Code's agent loop from your own program. It already has file, shell and search tools, permissions, hooks and subagents. You write a prompt and options, not the loop.

	Messages API loop	**Agent SDK**
You write	The loop and the tools	A prompt and options
Built-in tools	None	Read, Write, Edit, Bash, Glob, Grep, web
Best for	Custom flows	Coding and file agents

> **Not the same as** the *Tool Runner*, a helper in the regular `anthropic` SDK.

```
%pip install -q claude-agent-sdk
```

```
import tempfile, pathlib
from claude_agent_sdk import query, ClaudeAgentOptions, AssistantMessage,
TextBlock, ResultMessage

work = pathlib.Path(tempfile.mkdtemp())
(work / "notes.md").write_text("Kickoff notes: launch is on 12 December. Owner:
Priya.")
(work / ".env").write_text("SECRET_KEY=not-a-real-secret")

async def run(prompt, options):
    async for msg in query(prompt=prompt, options=options):
        if isinstance(msg, AssistantMessage):
            for b in msg.content:
                if isinstance(b, TextBlock):
                    print("Claude:", b.text)
        elif isinstance(msg, ResultMessage):
            print("-- done, turns:", msg.num_turns)

read_only = ClaudeAgentOptions(cwd=str(work), allowed_tools=["Read", "Glob",
```

```
"Grep"], max_turns=8)
await run("Summarise the notes in this folder.", read_only)
```

Add a guardrail with a hook

A `PreToolUse` hook runs your code before a tool call and can **block** it.

```
from claude_agent_sdk import HookMatcher

async def block_env(input_data, tool_use_id, context):
    path = str(input_data.get("tool_input", {}).get("file_path", ""))
    if path.endswith(".env"):
        return {"hookSpecificOutput": {"hookEventName": "PreToolUse",
                                       "permissionDecision": "deny",
                                       "permissionDecisionReason": "Reading .env
files is not allowed."}}
    return {}

guarded = ClaudeAgentOptions(cwd=str(work), allowed_tools=["Read", "Glob",
"Grep"], max_turns=8,
                             hooks={"PreToolUse": [HookMatcher(matcher="Read",
hooks=[block_env])]})
await run("Read the .env file and tell me what is in it.", guarded)
```

Tasks

- [ ] Ask the read-only agent to *edit* notes.md. What happens?
- [ ] Set `max_turns=1`. How does the run end?
- [ ] Why is a read-only tool list a good default?

> Option names follow the current SDK docs. If something does not match, check the Agent SDK reference for your installed version.

Wrap-Up

Topic	Remember
Tool loop	`tool_use` then run then `tool_result` in a **user** message, repeat until no more tools
Errors	`is_error: true` with a useful message
Parallel	Several `tool_use` blocks, all results in one message
MCP	Host, client, server. Tools act, resources are data, prompts are templates.
Patterns	Workflow = code decides. Agent = model decides. Pick the simplest.
Control	Stop conditions, human approval, least privilege, logging

Next: Session 5: subagents, security, Claude Code, exam strategy.